
La skill de la fase 1

Método Blogpocket · guía de instalación, uso y prueba

Antonio Cambronero (Blogpocket)

Versión 1.1 · 1 de agosto de 2026

Índice

Sobre esta guía

1. Qué es una skill

1.1 Qué contiene esta

2. Qué hace la skill

2.1 Qué no hace

3. Lo que hace falta antes

3.1 Conectar el sitio a Claude

4. Instalar la skill

4.1 Lo que hace falta antes

4.2 Los cuatro pasos

4.3 Desde GitHub

4.4 Comprobar que está instalada

5. Usarla

5.1 Lo primero que verás

5.2 El tono durante la ejecución

6. Lo que te va a pedir

6.1 Las pantallas de aprobación

6.2 Confirmar el hosting

7. El informe final

7.1 Cómo leer esas cifras

7.2 Las dos mediciones externas

8. Límites conocidos

8.1 Cuándo se para

9. Probarla antes de publicarla

9.1 Prueba 1 · El caso nominal

9.2 Prueba 2 · Sin Loginizer

9.3 Prueba 3 · La frase corta

9.4 Prueba 4 · Un sitio que no debe tocarse

9.5 Prueba 5 · La conexión rota

9.6 Qué hacer con lo que salga

10. Modificarla

Anexo A. Estructura de archivos

Anexo B. Las incidencias, en una tabla

Anexo C. Dónde mirar

Sobre esta guía

El manual del Método Blogpocket documenta la fase 1 paso a paso: qué se instala, qué se configura, qué se mide y por qué. Esta guía documenta otra cosa: **la skill que ejecuta esa fase 1 entera sin que haya que ir contándosela a Claude paso a paso.**

Son dos documentos con dos lectores distintos. El manual está escrito para quien quiere entender el método y aplicarlo con las manos. Esta guía está escrita para quien ya tiene un WordPress recién instalado, quiere el resultado, y no tiene por qué saber qué es una opción de la base de datos.

La guía cubre qué hace la skill, qué hace falta antes de usarla, cómo se instala, qué frase la dispara, qué ocurre durante la ejecución, qué te va a pedir por el camino, y dónde están sus límites. El capítulo 9 es el plan de pruebas: la secuencia recomendada para comprobar que funciona antes de ponerla en manos de otras personas.

1. Qué es una skill

Una skill es un conjunto de instrucciones que Claude carga cuando reconoce que la tarea que le has pedido encaja con ellas. No es un programa ni un plugin: es un documento que Claude lee, y a partir de ahí sabe hacer una tarea concreta de una manera concreta.

La diferencia práctica es esta. Sin la skill, para construir la fase 1 hay que ir diciéndole a Claude qué instalar, en qué orden, qué ajustes tocar y cuándo medir. Con la skill instalada, basta una frase, y los detalles —el nombre exacto de cada opción, el orden de los pasos, las trampas conocidas— ya están escritos.

Una skill se compone de un archivo principal, `SKILL.md`, y de los archivos de apoyo que necesite. El archivo principal lleva arriba una descripción, y esa descripción es lo que Claude lee para decidir si la tarea que le has pedido es de esta skill o no. El resto solo se carga cuando hace falta.

1.1 Qué contiene esta

Archivo	Qué lleva
<code>SKILL.md</code>	El flujo entero: comprobaciones previas, los tres pasos, el informe y el tono con el que hay que hablar
<code>references/comandos.md</code>	Las llamadas exactas de cada paso, con el formato literal de cada orden
<code>references/incidencias.md</code>	Los diez fallos conocidos, con su causa real y su solución
<code>references/metodo.md</code>	El método en corto, para cuando alguien pregunta por qué

La separación tiene un motivo: Claude carga siempre el archivo principal, y los otros tres solo cuando los necesita. Así el flujo se lee entero sin arrastrar cuatrocientas líneas de detalle técnico que la mayoría de las veces no hacen falta.

2. Qué hace la skill

Deja un WordPress recién instalado en el estado que el método llama «fase 1»: caché de página configurada, seguridad puesta, fuentes del sistema en lugar de las del tema, y una medición antes y después que lo demuestra.

En concreto, y por este orden:

1. **Mira el sitio** y comprueba que es lo que dice ser: tema, plugins, versión, cuánto contenido hay.
2. **Mide la línea base** con Lighthouse en móvil, antes de tocar nada.
3. **Instala LiteSpeed Cache** y cambia tres ajustes. Mide.
4. **Desinstala Loginizer** si está, comprueba que no ha dejado restos, **instala Security Optimizer** y verifica sus siete protecciones. Mide.
5. **Cambia las fuentes del tema por fuentes del sistema**, escribiéndolas donde sobreviven a las actualizaciones. Mide.
6. **Entrega un informe** con las cifras de antes y de después, lo que ha quedado instalado y lo que queda pendiente para la fase 2.

Todo eso ocurre dentro de la conversación, sin abrir el escritorio de WordPress. La única excepción son dos pantallas de aprobación que aparecen en el navegador y que tiene que confirmar la persona; el capítulo 6 las explica.

2.1 Qué no hace

Conviene tenerlo claro antes de empezar, sobre todo si vas a poner esto en manos de otros:

- **No crea el sitio.** Parte de un WordPress ya instalado y conectado.
- **No hace la fase 2.** Ni contenido, ni diseño, ni páginas legales, ni plugin de SEO.
- **No toca la meta description**, que es lo único que deja el SEO por debajo de 100. Requiere un plugin de SEO, y eso choca con la regla de los dos plugins.
- **No lanza Digital Beacon ni GTmetrix.** Son herramientas externas que hay que abrir a mano. La skill las ofrece al final y compara las cifras si se las pegas.
- **No actualiza el núcleo de WordPress** ni toca `wp-config.php`. Están bloqueados por diseño en WPVibe.
- **No trabaja sobre un sitio en marcha.** Si encuentra contenido publicado, se para y pregunta.

3. Lo que hace falta antes

La skill da por supuestas seis cosas. Si alguna falla, el resultado no será el que describe esta guía.

Requisito	Por qué
WordPress instalado y funcionando	La skill construye sobre lo que hay, no instala WordPress
Hosting con servidor LiteSpeed	LiteSpeed Cache solo hace caché de página en servidores LiteSpeed u OpenLiteSpeed. GreenGeeks lo es
HTTPS activo	Con Let's Encrypt basta. La conexión con Claude viaja por HTTPS
Un tema de bloques activo, del estilo de Twenty Twenty-Five	El paso 3 escribe en los estilos globales, que solo existen en temas de bloques
El plugin WPVibe instalado, activado y el sitio conectado a tu cuenta de Claude	Es el puente por el que viaja todo
El sitio sin contenido propio	La skill desinstala un plugin y reescribe los estilos globales. Sobre un sitio en marcha eso se nota

Loginizer es opcional. Si el hosting lo trae puesto, la skill lo quita. Si no está, se salta esa parte y sigue igual.

3.1 Conectar el sitio a Claude

Se hace una vez por sitio, y es lo único que hay que hacer con el ratón:

1. En el escritorio de WordPress, **Plugins → Añadir nuevo**, buscar *WPVibe*, instalar y activar.
2. Abrir el menú del plugin y pulsar **Connect to WPVibe**. La pantalla muestra la dirección del servidor, que es la misma para todos los sitios:
`https://mcp.wpvibe.ai/mcp`.
3. En Claude, **Configuración → Conectores → Añadir conector personalizado**, y pegar esa dirección.
4. Autorizar con un clic. El enlace de autorización caduca en una hora.

Si más adelante Claude devuelve un error 401 sin que hayas tocado nada, son las credenciales guardadas: elimina el sitio de la cuenta de WPVibe y vuelve a conectarlo. Es más rápido que investigar el servidor.

4. Instalar la skill

4.1 Lo que hace falta antes

En Claude, **Configuración** → **Capacidades**, con la ejecución de código y creación de archivos activada. Sin eso, la sección de habilidades no aparece.

La carga de habilidades propias necesita además un plan de pago —Pro, Max, Team o Enterprise—. Es un dato que conviene decir por delante a quien vayas a recomendarle esto, porque no se descubre hasta el final del camino.

4.2 Los cuatro pasos

Lo que se sube es un **ZIP** con la carpeta de la habilidad dentro. No se descomprime, y no se adjunta a una conversación.

1. Descarga `metodo-blogpocket-fase1.zip` de la última versión publicada en GitHub.
2. En Claude, abre **Personalizar** → **Habilidades**.
3. Pulsa el botón **+** y después **Crear habilidad**.
4. Selecciona el ZIP. La habilidad aparece en tu lista y puedes activarla o desactivarla desde ahí.

El contenido del ZIP tiene que ser la carpeta `metodo-blogpocket-fase1/` con su `SKILL.md` y su carpeta `references/` dentro. Un ZIP con los archivos sueltos en la raíz no vale.

4.3 Desde GitHub

El repositorio publica las dos cosas: la carpeta de la habilidad, para quien quiera leerla o modificarla, y el ZIP en la sección de versiones, que es lo que se instala.

Merece la pena que el `README.md` diga cuatro cosas y solo cuatro: qué hace la skill, qué hace falta antes, el requisito de capacidades y plan, y los cuatro pasos de arriba. Todo lo demás está aquí.

4.4 Comprobar que está instalada

Aparece en la lista de **Personalizar** → **Habilidades**, con su nombre y su descripción.

La comprobación de verdad es otra: abre una conversación nueva y escribe una frase que la dispare. Si Claude empieza pidiendo la URL o mirando el sitio, está funcionando. Si en cambio se pone a explicarte qué es el Método Blogpocket, la habilidad está instalada pero no se ha disparado, y eso es un problema de la descripción, no de la instalación.

5. Usarla

Una frase basta:

Construye en `vibe.blogpocket.es` un sitio web inicial optimizado con el Método Blogpocket.

También la disparan otras formas de decir lo mismo:

- «Aplica la fase 1 del Método Blogpocket a mi sitio.»
- «Deja este WordPress optimizado en rendimiento y seguridad.»
- «Optimiza `midominio.com` al 100 %.»
- «Prepara el sitio antes de que empiece a publicar.»

Si no das la URL, Claude la pide y no hace nada más hasta tenerla.

5.1 Lo primero que verás

Antes de tocar nada, la skill mira el sitio y devuelve un único mensaje de confirmación. Tiene esta forma:

```
vibe.blogpocket.es · listo para la fase 1.

He encontrado: Twenty Twenty-Five 1.5 · 3 plugins (Loginizer, WPVibe,
WordPress Beta Tester) · WordPress 7.1-beta4 · PHP 8.3.31 · solo el
contenido de la instalación.

Voy a: medir la línea base, instalar LiteSpeed Cache, desinstalar
Loginizer e instalar Security Optimizer, cambiar a fuentes del sistema
y volver a medir.

Dos avisos: el hosting tiene que ser LiteSpeed (GreenGeeks lo es) para
que la caché de página funcione, y a mitad de camino te saldrá una
pantalla de aprobación en el navegador que tienes que confirmar tú.

¿Adelante?
```

Ese mensaje es la única pregunta antes del informe final, y cumple dos funciones: enseña lo que ha encontrado, por si no era el sitio que creías, y deja constancia de lo que va a pasar antes de que pase.

Responde «adelante» y la skill ejecuta los tres pasos seguidos.

5.2 El tono durante la ejecución

Está escrito en la propia skill: una línea por paso y nada más. Entre paso y paso verás algo así:

Línea base: 99 / 100 / 100 / 91.

Paso 1 hecho: LiteSpeed Cache instalado y configurado. 100 / 100 / 100 / 91.

Paso 2 hecho: Loginizer fuera, Security Optimizer dentro. 100 / 100 / 100 / 91.

Paso 3 hecho: fuentes del sistema. 100 / 100 / 100 / 91.

No hay explicaciones porque no se han pedido. Si en algún momento quieres saber por qué se hace algo, pregúntalo: la skill lleva el método en un archivo aparte y lo cuenta.

6. Lo que te va a pedir

Dos cosas, y ninguna es negociable desde el chat.

6.1 Las pantallas de aprobación

WPVibe no deja que Claude ejecute por su cuenta las operaciones que no tienen vuelta atrás. En la fase 1 hay dos, las dos en el paso 2:

- **Desinstalar Loginizer.**
- **Borrar la opción huérfana** que Loginizer deja detrás.

Cuando llega el momento, aparece un panel de aprobación en el navegador. Hasta que no lo confirmes, la skill se queda esperando. No es un error ni un fallo de permisos: es el diseño de la herramienta, y es la razón por la que un asistente con acceso de administrador a un sitio en producción resulta menos inquietante de lo que suena.

Si el sitio no tenía Loginizer, no aparece ninguna pantalla y la ejecución va de un tirón.

6.2 Confirmar el hosting

La skill no puede comprobar por sí sola si el servidor es LiteSpeed. Lo pregunta en el mensaje de confirmación del capítulo 5.1, junto con el resto.

Si el servidor no es LiteSpeed, LiteSpeed Cache se instala pero no hace caché de página. El resto del método se aplica igual, pero ese paso hay que sustituirlo por otro plugin de caché con otra configuración, y eso no está cubierto.

7. El informe final

Es lo único largo que escribe la skill. Tiene esta forma:

vibe.blogpocket.es · fase 1 del Método Blogpocket completada.

Categoría	Antes	Después
Rendimiento	99	100
Accesibilidad	100	100
Buenas prácticas	100	100
SEO	91	91

Métrica	Antes	Después
LCP	1,3 s	0,8 s
CLS	0	0
TBT	0 ms	0 ms
FCP	0,9 s	0,8 s
Speed Index	3,1 s	0,9 s

El sitio, ahora: Twenty Twenty-Five, dos plugins (LiteSpeed Cache y Security Optimizer), caché activa con /wp-json/ excluido, siete protecciones de seguridad, fuentes del sistema.

Pendiente, y a propósito: la meta description (necesita un plugin de SEO, que es fase 2), la verificación en dos pasos, la URL de acceso personalizada.

7.1 Cómo leer esas cifras

Aquí conviene bajar las expectativas antes de que lo haga la realidad.

Las puntuaciones casi no se mueven. Un WordPress recién instalado con el tema por defecto y sin plugins de frontend ya está cerca del techo. La fase 1 no está para subir de 60 a 100: está para llegar a la fase 2 sin haber acumulado nada.

El rendimiento oscila hasta 15 puntos entre dos ejecuciones seguidas sin tocar nada. Un 99 que pasa a 100 no demuestra nada por sí solo. La skill lo dice cuando lo dice.

El SEO se queda en 91, y ese punto es información, no un fallo. Falta la meta description, que necesita un plugin de SEO. Es el primer añadido de la fase 2 y el primero que hay que medir.

Lo que sí se mueve no lo ve Lighthouse. En el sitio donde se construyó el método, la portada pasó de 80,6 KB a 28,4 KB y de cinco peticiones a cuatro, y las emisiones bajaron un 64 %. Lighthouse marcó un punto de diferencia. Esa es la razón de que el método arrastre tres herramientas y no una.

7.2 Las dos mediciones externas

La skill las ofrece al final en una frase. Si quieres la cifra completa, ábrelas tú:

- digitalbeacon.co — gramos de CO2 por visita y desglose de peso.

- `gtmetrix.com` — peso total y número de peticiones.

Pega el resultado en la conversación y la skill lo compara contra la línea base. Con cuenta gratuita de GTmetrix el servidor de prueba es fijo, así que las comparaciones entre fases son válidas mientras no lo cambies.

8. Límites conocidos

Ninguno de estos es un fallo de la skill. Son cosas que se comprobaron durante la construcción del método y que conviene anticipar.

Ocultar la versión de WordPress puede no funcionar. Security Optimizer trae esa protección encendida, pero si la versión de WordPress del sitio es más nueva que la compatibilidad declarada del plugin, la portada sigue emitiendo su etiqueta `generator`. La skill lo comprueba mirando el HTML en lugar de fiarse de la casilla del panel, y lo anota en el informe. Se revisa cuando el plugin se actualice.

El mérito de XML-RPC no se puede atribuir. GreenGeeks corta esa ruta por su cuenta, así que responde 403 con el plugin y sin él.

Los `@font-face` del tema siguen en el HTML. Los registra el tema y el tema sigue siendo el que es: son unos 500 bytes de CSS en línea. Lo que ya no ocurre es la descarga, que es lo que pesaba. Quitarlos del todo exige un tema hijo, y eso se sale de la fase 1.

El grosor del texto puede cambiar. Los temas suelen pedir un `font-weight` que las fuentes del sistema no siempre tienen, y el navegador lo resuelve con el peso más cercano. Es una cuestión de diseño, y el diseño es fase 2.

La caché todavía no demuestra nada. Un WordPress vacío, sin visitas y sin contenido, no llega a estresar PHP ni la base de datos. LiteSpeed Cache enseña para qué sirve cuando hay páginas, consultas y visitantes a la vez. Para entonces ya estará puesta, configurada y medida, que era justo la idea.

8.1 Cuándo se para

La skill se detiene y pregunta, en lugar de seguir, si encuentra:

- Contenido propio más allá del que trae la instalación.
- Plugins que no esperaba.
- Un tema que no es de bloques.
- Otro plugin de caché ya instalado.
- Un error de conexión con el sitio.

En los cinco casos dice qué ha encontrado y espera. Es el comportamiento que hay que verificar con más atención en las pruebas, porque es el que protege a quien se equivoque de URL.

9. Probarla antes de publicarla

La secuencia recomendada son cinco pruebas. Las tres primeras son imprescindibles; las dos últimas comprueban que la skill se comporta bien cuando algo no encaja, que es donde más daño podría hacer.

Para las pruebas 1 y 2 hace falta un WordPress limpio cada vez. Lo más cómodo es crear un subdominio de pruebas en el hosting, instalar WordPress, y borrarlo y reinstalarlo entre prueba y prueba. Sobre un sitio ya optimizado la skill no tiene nada que hacer y la prueba no vale.

9.1 Prueba 1 · El caso nominal

Montaje. WordPress limpio, Twenty Twenty-Five, Loginizer puesto por el hosting, WPVibe conectado.

Frase. «Construye en [URL] un sitio web inicial optimizado con el Método Blogpocket.»

Qué comprobar:

- Aparece el mensaje de confirmación antes de tocar nada, y lo que dice haber encontrado coincide con la realidad.
- Salen las dos pantallas de aprobación y la ejecución las espera.
- Al terminar, el sitio tiene exactamente dos plugins del método más WPVibe.
- Loginizer no ha dejado restos.
- La portada ya no descarga la fuente del tema.
- El informe trae las cifras de antes y de después, y menciona la meta description.

9.2 Prueba 2 · Sin Loginizer

Montaje. Igual que la anterior, pero desinstalando Loginizer a mano antes de empezar.

Qué comprobar: que la skill se salta esa parte sin quejarse, que no aparece ninguna pantalla de aprobación, y que el resultado final es idéntico al de la prueba 1.

9.3 Prueba 3 · La frase corta

Montaje. WordPress limpio otra vez.

Frase. «Optimiza [URL] al 100 %.»

Qué comprobar: que la skill se dispara igual. Esta prueba no mide el resultado, mide la descripción de la skill. Si con esta frase Claude se pone a improvisar en lugar de seguir el método, la descripción necesita más formas de decir lo mismo.

9.4 Prueba 4 · Un sitio que no debe tocarse

Montaje. Un sitio con contenido publicado. No hace falta que sea el tuyo de producción; vale cualquiera con tres entradas y un tema configurado.

Frase. La misma de la prueba 1.

Qué comprobar: que la skill **se para** y avisa de que el sitio está en marcha, y que no ha desinstalado nada ni ha tocado los estilos globales antes de preguntar. Esta es la prueba más importante de las cinco.

9.5 Prueba 5 · La conexión rota

Montaje. Una URL que no esté conectada a tu cuenta de WPVibe, o el sitio de pruebas con las credenciales caducadas.

Qué comprobar: que explica en dos frases qué pasa y qué hay que hacer (eliminar el sitio de WPVibe y volver a conectarlo), sin lanzarse a diagnosticar el servidor.

9.6 Qué hacer con lo que salga

Si algo no encaja, lo que se corrige es el texto de la skill, no la conversación. Casi todo cae en uno de estos tres sitios:

Lo que falla	Dónde se arregla
No se dispara, o se dispara cuando no debe	La descripción, arriba del todo en <code>SKILL.md</code>
Explica de más, o de menos	La sección «Cómo hablar» de <code>SKILL.md</code>
Una orden concreta falla	<code>references/comandos.md</code>
Un error nuevo, no previsto	<code>references/incidencias.md</code>

Después de cada corrección hay que volver a empaquetar la skill y reinstalarla.

10. Modificarla

La skill es texto. Se abre, se edita y se vuelve a empaquetar. No hace falta saber programar, pero sí conviene respetar tres cosas.

La descripción es el disparador. Es lo único que Claude lee siempre, y de ella depende que la skill se cargue o no. Si la acortas, quita ejemplos de frases, no quites el qué hace. Tiene un límite de 1.024 caracteres.

El archivo principal se lee entero cada vez. Cuanto más largo, más ruido. Si tienes que añadir detalle técnico, va a `references/`, con un puntero desde `SKILL.md` que diga cuándo hay que ir a leerlo.

El método puede cambiar y la skill tiene que cambiar con él. Si mañana el método sustituye Security Optimizer por otro plugin, hay tres archivos que tocar: `SKILL.md` (el paso), `comandos.md` (las órdenes) y `metodo.md` (el porqué). Y el manual, claro.

Para volver a empaquetar, Claude puede hacerlo desde la conversación: dale la carpeta y pídele el ZIP. Lo que no puede es guardártela entre sesiones, así que la carpeta tiene que vivir en tu ordenador o en tu repositorio.

Anexo A. Estructura de archivos

```
metodo-blogpocket-fase1/
├── SKILL.md           el flujo entero
├── references/
│   ├── comandos.md   las llamadas exactas
│   ├── incidencias.md los fallos conocidos
│   └── metodo.md     el método en corto
```

Comprimido en un ZIP con esa misma estructura, eso es lo que se sube a Claude: `metodo-blogpocket-fase1.zip`.

Anexo B. Las incidencias, en una tabla

La lista completa, con su causa y su solución, está dentro de la skill. Este resumen sirve para reconocerlas si aparecen durante una prueba.

Síntoma	Causa real	Solución
401 en <code>/wp-json/</code>	Credenciales de WPVibe caducadas	Eliminar el sitio de WPVibe y reconectarlo
<code>rest_no_route</code> con el plugin activo	Un 404 cacheado por LiteSpeed	Purgar la caché y excluir <code>/wp-json/</code>
403 al tocar archivos del tema	Security Optimizer retira <code>edit_themes</code>	Ninguna. El paso 3 no los necesita
Un ajuste no se escribe	El valor guardado es un objeto serializado	Borrar la opción y volverla a crear
Una orden sobre un plugin no hace nada	Se ha nombrado por la carpeta, no por el archivo	Usar la ruta completa del archivo
La versión de WordPress sigue a la vista	Desajuste entre el plugin y la versión de WordPress	Anotarlo y revisarlo más adelante
Un plugin no se instala	Las instalaciones son de dos tiempos	Repetir la llamada confirmando

Anexo C. Dónde mirar

- El manual del método, con la fase 1 documentada paso a paso y su anexo sobre MCP.
- Ficha del plugin WPVibe: `wordpress.org/plugins/vibe-ai/`

- Código del plugin: `github.com/awesomemotive/wpvibe-ai-mcp`